

PRACTICAL-6

QNS: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
GNU nano 8.7.1 server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define REQUEST_FIFO "request_fifo"
#define RESPONSE_FIFO "response_fifo"

int main()
{
    char message[100];
    char response[150];
    mkfifo(REQUEST_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

    printf("Server started...\n");
    printf("Waiting for clients...\n");

    while (1)
    {
        int request_fd = open(REQUEST_FIFO, O_RDONLY);

        if (request_fd == -1)
        {
            perror("Error opening request FIFO");
            exit(1);
        }

        ssize_t bytes_read = read(request_fd, message, sizeof(message) - 1);
        close(request_fd);

        if (bytes_read > 0)
```

```
    if (request_fd == -1)
    {
        perror("Error opening request FIFO");
        exit(1);
    }
    ssize_t bytes_read = read(request_fd, message, sizeof(message) - 1);
    close(request_fd);

    if (bytes_read > 0)
    {
        message[bytes_read] = '\0';

        printf("Client sent: %s\n", message);
        snprintf(response, sizeof(response),
            "Server processed: %s", message);

        int response_fd = open(RESPONSE_FIFO, O_WRONLY);

        if (response_fd == -1)
        {
            perror("Error opening response FIFO");
            exit(1);
        }

        write(response_fd, response, strlen(response) + 1);

        close(response_fd);
    }
}

return 0;
}
```

```
priylvn@LAPTOP-NOJKE600:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

```
priylvn@LAPTOP-NOJKE600:~$ mkdir fifo_project
cd fifo_project
priylvn@LAPTOP-NOJKE600:~/fifo_project$ pwd
/home/priylvn/fifo_project
priylvn@LAPTOP-NOJKE600:~/fifo_project$ nano server.c
priylvn@LAPTOP-NOJKE600:~/fifo_project$ ls
server.c
priylvn@LAPTOP-NOJKE600:~/fifo_project$ gcc server.c -o server
priylvn@LAPTOP-NOJKE600:~/fifo_project$ nano server.c
```

```
The client-server communication was successfully performed using Named Pipes (FIFOs). Messages sent by the client were received and p
rocessed by the server, and responses were sent back through the FIFO. When multiple clients communicated simultaneously, t
priylvn@LA
PTOP-NOJKE600:~/fifo_project$ The client-server communication was successfully performed using Named Pipes (FIFOs). Messages sent by
the client were received and processed by the server, and responses were sent back through the FIFO. When multiple clients communicat
ed simultaneously, the common FIFO handled their messages in an ordered manner, but a shared response FIFO could cause response-match
ing issues.
```

OBSERVATION:

The client-server communication was successfully performed using Named Pipes (FIFOs). Messages sent by the client were received and processed by the server, and responses were sent back through the FIFO. When multiple clients communicated simultaneously, the common FIFO handled their messages in an ordered manner, but a shared response FIFO could cause response-matching issues.

QNS: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
    if (signal == SIGINT)
    {
        printf("\nSIGINT received! (Ctrl+C)\n");
    }
    else if (signal == SIGTERM)
    {
        printf("\nSIGTERM received! Termination requested.\n");
    }
    else if (signal == SIGUSR1)
    {
        printf("\nSIGUSR1 received! User-defined signal handled.\n");
    }
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process started.\n");
    printf("Process ID (PID): %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

```
priylvn@LAPTOP-NOJKE600:~$ cd fifo_project
priylvn@LAPTOP-NOJKE600:~/fifo_project$ nano signals.c
priylvn@LAPTOP-NOJKE600:~/fifo_project$ gcc signals.c -o signals
priylvn@LAPTOP-NOJKE600:~/fifo_project$ ls
server  server.c  signals  signals.c  signals.c.save
priylvn@LAPTOP-NOJKE600:~/fifo_project$ ./signals
Process started.
Process ID (PID): 925
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received! (Ctrl+C)
```

OBSERVATION:

The POSIX signal handling program was successfully executed in Ubuntu. The program registered handlers for **SIGINT**, **SIGTERM**, and **SIGUSR1**. When **SIGINT** was generated using Ctrl+C, the signal handler displayed a message instead of terminating the program. The program continued executing, demonstrating **asynchronous event handling**. Similarly, SIGTERM and SIGUSR1 can be sent using the kill command and are handled by the registered signal handler. This demonstrates how signal handlers allow a process to respond to events asynchronously while it is running.